

Documentação do projeto UFSKernel

Kernel: o Núcleo do Sistema Operacional

Comunidade UFSKernel

31 de agosto de 2026

Índice

1	Infraestrutura Base	4
1.1	Descrição	4
1.2	Decisões de Arquitetura e Design	4
1.3	Detalhes da Implementação	4
1.3.1	Código de inicialização	4
1.3.2	<i>Linker script</i>	5
1.3.3	Código em C	6
1.4	Interface Pública (API)	7
2	Entrada e Saída de teclado	8
2.1	Descrição	8
2.2	Decisões de Arquitetura e Design	8
2.3	Detalhes da Implementação	9
2.3.1	Arquivos e Organização	9
2.3.2	Fluxo Simplificado de Execução	9
2.3.3	A Entrada de Dados via Teclado	10
2.3.4	Inicialização da UART	10
2.3.5	Tratamento de Interrupções	10
2.3.6	Gerenciamento de <i>Buffer</i> Circular	12
2.3.7	A Saída de Dados no Monitor	12
2.4	Interface Pública (API)	12
2.4.1	API da UART (Baixo Nível)	13
2.4.2	API de Entrada e Saída do Kernel (<i>kstdio</i>)	13
2.5	Testes e Verificação	14
3	Memória	16
3.1	Descrição	16
3.2	Decisões de Arquitetura e Design	16
3.3	Detalhes da Implementação	17
3.3.1	Gerenciador de Memória Física (PMM)	17
3.3.2	Gerenciador de Memória Virtual (VMM)	18
3.3.3	API de Memória	18
3.4	Interface Pública (API)	19
3.4.1	API de Uso Geral (Gerenciamento de Heap)	19
3.4.2	API de Baixo Nível / Core (Subsistemas Críticos)	19
3.5	Testes e Verificação	20
4	Chamadas de Sistema	22
4.1	Descrição	22
4.2	Decisões de Arquitetura e Design	22
4.3	Detalhes da Implementação	23
4.4	Interface Pública (API)	24
4.4.1	<code>printf</code>	24
4.4.2	<code>abort</code>	24
4.5	Testes e Verificação	24

5	Processos, Troca de Contexto e Escalonamento	25
5.1	Descrição	25
5.2	Fundamentação Teórica	25
5.2.1	Processo e PCB	25
5.2.2	Trapframe, Contexto de Kernel e ABI ARM	25
5.2.3	Escalonamento: Round-Robin e MLFQ	26
5.3	Módulo de Processos	26
5.3.1	Decisões de Arquitetura e Design	26
5.3.2	Detalhes da Implementação	27
5.4	Mecanismo de Troca de Contexto	28
5.4.1	Três mecanismos: entrada, retorno e troca	28
5.4.2	Entrada via SVC e IRQ	28
5.4.3	Retorno ao Modo Usuário	29
5.4.4	Troca de Contexto de Kernel	29
5.5	Escalonador	29
5.5.1	Decisões de Arquitetura e Design	29
5.5.2	Detalhes da Implementação	30
5.6	Interface Pública (API)	31
5.6.1	Módulo de Processos (<code>process.h</code>)	31
5.6.2	Módulo do Escalonador (<code>scheduler.h</code>)	31
5.7	Testes e Verificação	32
5.7.1	Rotina de Teste	32
5.7.2	Saída Esperada	32
6	Tratador de Interrupções	33
6.1	Descrição	33
6.2	Decisões de Arquitetura e Design	33
6.3	Detalhes da Implementação	33
6.4	Interface Pública (API)	35
6.5	Testes e Verificação	36

Infraestrutura Base

Autor(es): *Ti Ribeiro Silvério*

1.1 Descrição

Esse é o kernel mínimo para possibilitar o desenvolvimento dos módulos subsequentes, um executável para o QEMU ARM que coloca o processador para executar código C compilado. O código aceito é extremamente limitado, contando apenas com funções simples de impressão de constantes na porta serial do QEMU, que é redirecionada para a saída padrão da máquina hospedeira.

1.2 Decisões de Arquitetura e Design

Para o desenvolvimento deste projeto, foi adotada uma filosofia de simplicidade máxima. Nele, foi feito o mínimo possível para que o processador ARM virtual executasse código C compilado. Essa escolha foi feita para tornar o entendimento dos detalhes mais fácil, e possibilitar que funcionalidades extras sejam adicionadas à medida que forem necessárias.

Um exemplo disso foi a preparação da pilha apontada pelo registrador `sp`. Para solucionar esse problema, foi alocado um espaço de 16KB no binário final dentro do *linker script*. Porém, por mais que tenha sido definido um rótulo que armazena a posição de memória do fim da pilha, não foi implementada uma maneira de tratar o estouro. Dessa forma, essas soluções serão implementadas em projetos futuros à medida que problemas aparecerão.

1.3 Detalhes da Implementação

Esse projeto consiste em 4 partes:

1. O código de inicialização em Assembly ARM;
2. O *linker script* que liga o código de inicialização ao código em C compilado e prepara a pilha;
3. A função principal em C para a qual o processador é desviado no código de inicialização;
4. A API driver do serial virtual do QEMU que é usada como saída padrão, chamada na função principal.

1.3.1 Código de inicialização

O código em Assembly é o mais simples de todos. Primeiro, ele coloca o processador no modo supervisor e desliga interrupções, depois configura a pilha e desvia a execução para o código em C.

```

1  _start:
2      mrs r0, cpsr
3      bic r0, r0, #0x1f
4      orr r0, r0, #0xd3
5      msr cpsr, r0
6
7      ldr sp, =stack_top
8      bl kmain
9
10 hang:
11     b hang

```

Listing 1.1: Configuração do modo do processador da pilha e chamada a `kmain()`.

Nesse código, o registrador `r0` é usado como registrador auxiliar para executar as mudanças. Assim, é escrito 11010011_2 no último byte do registrador `cpsr`, que armazena o estado atual do modo de operação do processador. Os dois bits mais a esquerda fazem o processador ignorar interrupções, enquanto os cinco mais a direita ligam o modo supervisor [1].

É importante desativar as interrupções pois ainda não definimos como tratá-las. Caso acontecesse qualquer interrupção durante a execução de um programa, sua execução seria desviada para uma posição de memória desconhecida e isso travaria o sistema.

Também é essencial ligar o modo supervisor, pois o kernel deve ser capaz de ter controle total sobre o computador e sua memória, diferente dos demais processos que rodarão no modo usuário.

Após isso, é configurada a pilha e desviado o contador de programa para o binário da função `kmain()`. Caso haja um retorno, o processador entra em um laço infinito para impedir a execução de lixo de memória.

A inicialização do *stack pointer* é de extrema importância, pois chamadas de funções e passagem de parâmetros são compiladas para saltos e empilhamento de valores. Sem a existência da pilha, não seria possível chamar funções em C.

1.3.2 *Linker script*

Geralmente, quando usamos o *linker* para terminar de gerar um executável, nem nos preocupamos com *como* os binários serão ligados. No caso do kernel, esse não é o caso, e precisamos organizar as seções dos binários a serem ligados de tal forma que sejam executadas corretamente durante o *boot*. Para resolver esse problema, temos que definir essas especificidades em uma receita para o *linker*.

Esse *script*, assim como o código de inicialização, é bem simples.

```

1 ENTRY(_start)
2
3 SECTIONS {
4     . = 0x40000000;
5
6     .text : {
7         KEEP(*(.text.startup))
8         *(.text)
9         *(.rodata)
10    }
11
12    .data : {
13        *(.data)
14    }
15
16    .bss : {
17        *(.bss)

```

```

18     }
19
20     stack_bottom = .;
21     . = . + 0x4000;
22     stack_top = .;
23
24     end = .;
25 }

```

Listing 1.2: Script de geração do binário final.

A primeira linha é responsável por apontar que o início lógico do executável deve ser `_start`, o rótulo definido no código Assembly.

O bloco da linha 3 é a definição da ordem dos binários a serem ligados no binário final. Nesse bloco, definimos o início da DRAM (endereço `0x40000000` [2]) e o que vai dentro de cada seção do executável: `.text`, `.data` e `.bss`.

A seção `.text` é onde deve entrar o código fonte do programa, pois é uma seção com permissões de leitura e execução. Nela, colocamos o código de iniciação da subseção anterior, as seções `.text` de cada binário a ser ligado e as seções `.rodata` de cada um deles.

Essa ultima seção (`.rodata`) não armazena código, mas sim valores constantes, como cadeias de caracteres literais como em `print("Hello World")`. A constante "Hello World" não é uma variável, mas ainda precisa ser armazenada em algum lugar, nesse caso, `.rodata`. Como `.text` é uma seção de apenas leitura, é um bom lugar para armazenar esse tipo de dado.

Similarmente, as seções `.data` e `.bss`, que contém as variáveis globais, são preenchidas com a junção das seções de mesmo nome dos arquivos objeto.

Por fim, vem a definição das posições de início e fim da pilha e a alocação de seu espaço. O fim¹ da pilha é definido como o endereço logo após a seção `.bss`, são pulados 16KB, e o início da pilha é endereço após esse salto. Dessa forma, reservamos um espaço para que a pilha possa funcionar normalmente.

1.3.3 Código em C

O código em linguagem de alto nível é dividido em três arquivos: `include/serial.h`, `kernel/serial.c` e `kernel/main.c`.

```

1 #define UART0_DR ((volatile unsigned int *)0x09000000)
2
3 void serial_putc(char c) { *UART0_DR = c; }
4
5 void serial_puts(const char *s) {
6     while(*s) serial_putc(*s++);
7 }
8
9 void serial_init(void) {}

```

Listing 1.3: Implementação das funções driver serial.

Esse é o código responsável por exibir dados na saída do serial. Inicialmente, é definido² o endereço de memória mapeado para “conversar” com o dispositivo de saída. A porta serial do QEMU funciona com E/S mapeada em memória, então o dispositivo, sempre que nota uma mudança em seu endereço mapeado, dá uma resposta.

¹Observação: a pilha “cresce para baixo”, com o topo começando em um valor alto e decrescendo até um valor mais baixo. Decidi, por clareza, chamar de “fim” o ponto mais baixo, o limite de decrescimento, e de “início” o ponto mais alto, onde ela começa vazia.

²É importante notar que esse endereço não é fixo, como dito na documentação [2]. No **futuro**, devemos obter esse endereço através do *Device Tree Blob*.

Como consequência disso, enviar dados ao serial é tão simples quanto alterar o conteúdo de uma posição de memória, sendo esta pré-definida com o `#define` no topo do arquivo. Foi usado o tipo `volatile` para evitar que o compilador faça otimizações indesejadas de leitura e escrita, uma vez que o conteúdo dessa posição pode mudar a qualquer momento.

```
1 #include <serial.h>
2
3 void kmain(void) {
4     serial_init();
5     serial_puts("Bem-vindo ao UFSKernel!\n");
6     serial_puts("Executando em modo ARM bare-metal no QEMU.\n");
7
8     return;
9 }
```

Listing 1.4: Função principal.

Por fim, todo o programa é testado com a impressão de texto na porta serial. Como executamos o QEMU com a *flag* `-nographic`, a saída da porta serial é redirecionada para a saída padrão do terminal que executou o QEMU.

1.4 Interface Pública (API)

As únicas funções de API criadas nesse projeto foram `serial_putc()` e `serial_puts()`. Para utilizá-las, basta incluir a biblioteca `serial.h` que já é biblioteca padrão do UFSKernel. Elas recebem, respectivamente, um `char` e uma `string` constante, e enviam esses dados para a porta serial.

Essas funções serão essenciais para o desenvolvimento dos próximos projetos, tanto na execução de testes quanto durante a depuração do código.

Entrada e Saída de teclado

Autor(es): *Matteo Cileneo Savan, Pedro Achilles Domingues Alves Pereira e Raul Yoshiyuki Komai*

2.1 Descrição

A entrada e saída (E/S) de dados formam a base da utilidade de qualquer sistema computacional. Sem esses subsistemas, a máquina torna-se incapaz de se comunicar com o mundo exterior. Um computador isolado é, na prática, apenas um amontoado de silício incapaz de resolver problemas reais. Neste contexto, esta seção detalha a implementação dos mecanismos de E/S, com foco na captura de dados via teclado e na saída visual em tela.

Tradicionalmente, a arquitetura dos dispositivos de E/S pode ser dividida em dois componentes principais:

Componente Físico (ou Mecânico) o dispositivo em si. É o hardware responsável por executar a operação de movimentar, exibir ou capturar os dados (as teclas de um teclado ou o painel de um monitor);

Componente Eletrônico a controladora de dispositivo, que atua como interface entre o sistema principal e o componente físico. Na prática, ela expõe registradores para a comunicação com a CPU; a partir desses registradores, o sistema operacional torna-se capaz de enviar e receber dados, ler o estado do dispositivo, entre outras operações de controle.

O foco da implementação deste módulo é interagir com as controladoras de dispositivo por meio da criação de um *driver* orientado a interrupções, eliminando a ineficiência da espera ocupada. Além disso, abordaremos o gerenciamento de *buffers* de E/S e a implementação de interatividade com o usuário por meio de um *shell* básico para a execução de comandos.

No ambiente do QEMU, a controladora simulada é a PrimeCell UART (PL011), que serve como base para a comunicação serial entre os dispositivos de E/S e o *kernel*. É importante ressaltar que, neste cenário, não ocorre comunicação direta da UART com o teclado físico, tampouco com o monitor; toda essa operação é intermediada pelo próprio emulador QEMU [3]. Dessa forma, a plataforma expõe a interface necessária para a manipulação da controladora em baixo nível.

2.2 Decisões de Arquitetura e Design

O projeto visa tornar o sistema mais modular e facilitar futuras manutenções. Para refletir essa decisão estrutural, a implementação foi dividida em dois módulos: o arquivo `uart.c` concentra as funções de baixo nível da controladora UART, enquanto o arquivo `kstdio.c` fornece uma camada de abstração de mais alto nível, focada nas rotinas de E/S.

A respeito do gerenciamento temporário de dados na implementação da UART, optou-se pela utilização de um *buffer* circular. Essa estrutura de dados foi escolhida por permitir o armazenamento e o consumo contínuo de fluxos de dados de maneira eficiente, operando sobre

um espaço de memória estático e pré-allocado, além de apresentar uma implementação direta e de baixa complexidade.

Por fim, a arquitetura de comunicação com os dispositivos baseia-se estritamente em um sistema orientado a interrupções. Essa decisão evita o desperdício de recursos computacionais: o processador fica livre para executar outros processos ou tarefas do sistema, sendo notificado pelo hardware apenas quando ocorre, de fato, um evento (como o pressionamento de uma tecla), eliminando a ineficiência da espera ocupada.

2.3 Detalhes da Implementação

2.3.1 Arquivos e Organização

A base de código foi estruturada nos seguintes arquivos:

kstdio.c camada de abstração de alto nível sobre as funcionalidades da UART. Fornece um conjunto de rotinas de entrada e saída baseadas nas funções primitivas de **uart.c**, sendo amplamente utilizada no interpretador de comandos do sistema;

main.c ponto de entrada do *kernel*, coordena a inicialização geral do sistema;

shell.c *shell* interativo básico, capaz de ler, interpretar e executar comandos do usuário a partir do terminal do QEMU;

uart.c encapsula toda a lógica de implementação do *driver* da controladora UART emulada.

2.3.2 Fluxo Simplificado de Execução

Podemos dividir a execução das operações de E/S em dois fluxos principais: o de entrada (leitura) e o de saída (escrita). A seguir, detalhamos o caminho percorrido pelos dados ao atravessarem as diferentes camadas de abstração do sistema.

Fluxo de Entrada

Neste fluxo, ilustrado na Figura 2.1, o dado nasce no mundo físico e viaja até ser processado pelo sistema operacional. O usuário digita caracteres no teclado, que são capturados pelo terminal do QEMU e encaminhados para a UART emulada. Ao receber os dados, a UART aciona uma interrupção de hardware. O *driver* intercepta essa interrupção, extrai os caracteres e os armazena de forma segura em um *buffer* de entrada, disponibilizando-os para o *shell* por meio de funções de leitura, como `uart_getc()`.

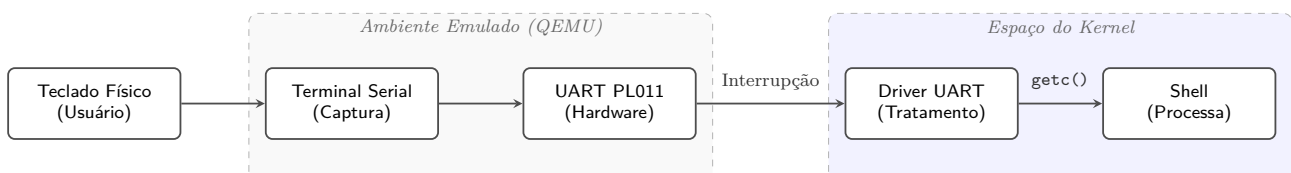


Figura 2.1: Pipeline do fluxo de entrada.

Fluxo de Saída

No fluxo de saída, ilustrado na Figura 2.2, o caminho se inverte. O *shell* utiliza funções da camada de abstração (como `kprintf()`, `kputs()` e `kputc()`) para enviar respostas ao *driver* da UART. O *driver*, por sua vez, escreve esses caracteres nos registradores da controladora emulada, que os transmite de volta ao QEMU para serem finalmente renderizados no terminal do usuário.

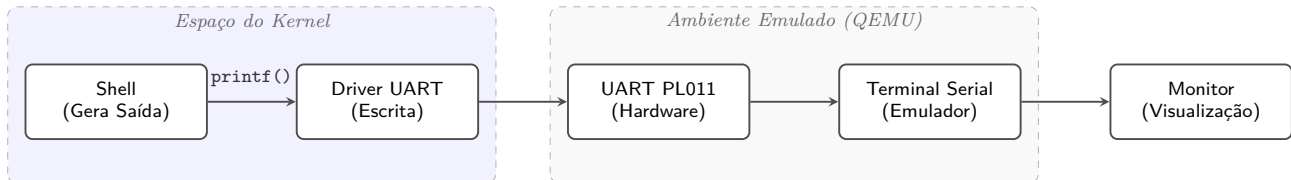


Figura 2.2: Pipeline do fluxo de saída.

2.3.3 A Entrada de Dados via Teclado

O teclado é um dispositivo de caractere. Diferentemente dos discos, que operam lendo e escrevendo dados em blocos de tamanho fixo, o teclado gera um fluxo sequencial e contínuo de caracteres independentes.

O driver implementado para gerenciar esse fluxo foi criado com base na especificação técnica da interface PL011 [4]. A comunicação em baixo nível é realizada mapeando na memória os principais registradores da controladora [5], conforme detalhado na Tabela 2.1.

O *kernel* efetiva o controle da UART lendo o *status* e escrevendo configurações e dados diretamente nestes endereços mapeados.

2.3.4 Inicialização da UART

A rotina de inicialização ocorre por meio da chamada `uart_init()`, que executa a seguinte sequência de passos, ilustrada na Figura 2.3.

Ao final deste processo, o sistema encontra-se devidamente configurado para operar de forma assíncrona. Ao dispensar a espera ocupada (*polling*), o *kernel* delega ao controlador de interrupções a tarefa de notificá-lo apenas quando houver, de fato, novos caracteres a serem processados.

2.3.5 Tratamento de Interrupções

Sempre que um caractere é inserido no terminal do QEMU, a UART atualiza seu estado interno e levanta um sinal de interrupção (*IRQ*). O controlador de interrupções da arquitetura intercepta esse sinal e desvia o fluxo de execução para a rotina de tratamento `uart_irq_handler(void)`, que realiza os seguintes passos lógicos:

1. Verifica e valida a origem da interrupção lendo o registrador `UART0_MIS`;
2. Confirma o tratamento do evento escrevendo no registrador de limpeza `UART0_ICR`;
3. Consome os caracteres acumulados lendo diretamente o registrador `UART_DR`;
4. Armazena em memória cada caractere recuperado chamando `uart_buffer_push()`, em um laço que se repete enquanto o hardware indicar que há dados válidos pendentes de leitura.

Registrador	Nome Completo	Descrição Funcional
UART_CR	<i>Control Register</i>	Habilita ou desabilita a UART, bem como as vias de transmissão (TX) e recepção (RX).
UART_DR	<i>Data Register</i>	O <i>kernel</i> lê deste endereço para obter dados recebidos e escreve nele para transmitir novos dados.
UART_FR	<i>Flag Register</i>	Reflete o estado atual. Contém a flag RXFE (<i>Receive FIFO Empty</i>), que indica se a fila de recepção está vazia.
UART_LCR_H	<i>Line Control Register</i>	Controle de linha. Contém o bit FEN (<i>FIFO Enable</i>) para habilitar as FIFOs de hardware.
UART_IMSC	<i>Interrupt Mask</i>	Define eventos geradores de interrupção. Destaque para as máscaras RXIM (recepção) e TXIM (transmissão).
UART_ICR	<i>Interrupt Clear</i>	Registrador de escrita; informa ao hardware que uma interrupção pendente foi devidamente tratada.
UART_MIS	<i>Masked Int. Status</i>	Registrador de leitura; indica exatamente quais interrupções (não mascaradas) estão pendentes no momento.
IBRD e FBRD	<i>Baud Rate Divisor</i>	Contêm, respectivamente, as partes inteira e fracionária do divisor para configurar a taxa de transmissão.

Tabela 2.1: Principais registradores da controladora UART PL011 utilizados na comunicação de E/S.

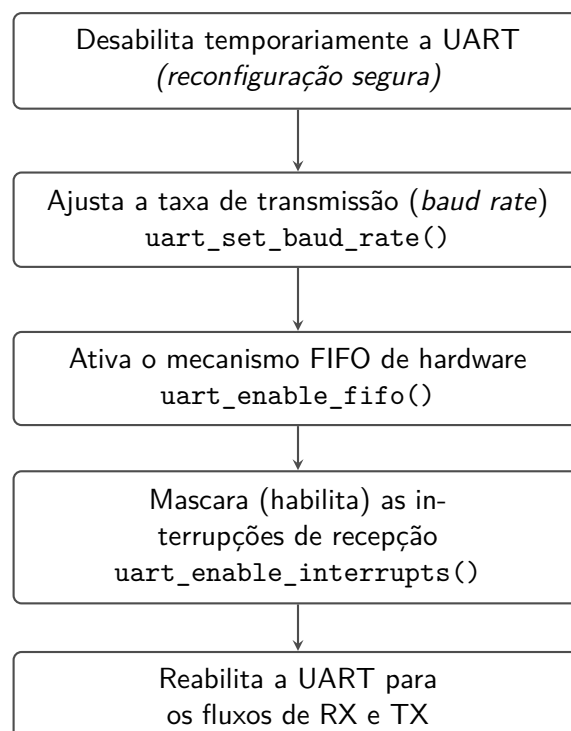


Figura 2.3: Sequência de inicialização da UART executada por `uart_init()`.

2.3.6 Gerenciamento de *Buffer* Circular

Como as velocidades de digitação e de processamento do *kernel* são assimétricas, é necessário o armazenamento temporário dos dados recebidos. Para isso, estruturou-se uma fila circular, definida pelo tipo `ring_buffer_t`, cujos campos estão descritos na Tabela 2.2.

Campo	Tipo	Descrição
<code>buffer</code>	<code>char[]</code>	Vetor estático de caracteres em memória.
<code>head</code>	<code>volatile uint32_t</code>	Índice que rastreia a posição da próxima escrita.
<code>tail</code>	<code>volatile uint32_t</code>	Índice que rastreia a posição da próxima leitura.

Tabela 2.2: Campos da estrutura `ring_buffer_t`.

A Figura 2.4 ilustra o funcionamento do *buffer* circular:

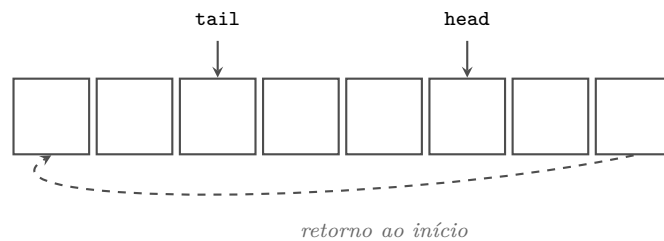


Figura 2.4: Os índices `head` (escrita) e `tail` (leitura) avançam de forma independente e retornam ao início do vetor ao atingir a última posição.

A partir dessa estrutura central, instancia-se o objeto `rx_buffer` (dedicado à recepção), cujas operações de inserção e remoção ocorrem estritamente por meio da interface de funções do *driver*:

static int uart_buffer_push(char c) insere o caractere `c` na posição `head` e avança o índice circularmente. Retorna 1 em caso de sucesso (havendo espaço livre) e 0 caso contrário;

static int uart_buffer_pop(char *ptr) consome um caractere da posição `tail`, gravando-o na região de memória apontada por `ptr`. Retorna 1 se houver dado disponível e 0 se o *buffer* estiver vazio.

2.3.7 A Saída de Dados no Monitor

Assim como o teclado, o monitor de texto é gerido pelo sistema como um dispositivo de caractere. No ecossistema emulado deste projeto, as saídas textuais fluem pela mesma controladora UART (PL011). Toda vez que o *kernel* escreve um caractere para exibição, o hardware transmite essa informação de volta pela interface serial, encarregando o QEMU de renderizar os caracteres no terminal de saída visual.

2.4 Interface Pública (API)

O subsistema de E/S disponibiliza duas *APIs*: uma de baixo nível, voltada ao acoplamento direto com o hardware, e outra de alto nível, estruturada para abstrair operações comuns ao

restante do *kernel*.

2.4.1 API da UART (Baixo Nível)

Esta camada agrupa as funções primitivas de interação direta com a controladora UART PL011. Suas principais rotinas são:

void uart_init(void) Inicializa e configura a UART PL011 para comunicação serial. Durante o processo, a controladora é temporariamente desabilitada, a taxa de transmissão (*baud rate*) é ajustada, os *buffers* FIFO são ativados e as interrupções de recepção são habilitadas. Ao término, a UART é reativada para operações de transmissão e recepção.

void uart_putc(char c) Transmite um único caractere *c* pela UART. Realiza automaticamente o tratamento do caractere de nova linha (`\n`), convertendo-o para a sequência (`\r\n`) exigida por terminais seriais. Antes de efetuar a escrita no registrador de dados UART_DR, a função aguarda a liberação de espaço no FIFO de transmissão.

void uart_puts(const char *str) Transmite uma sequência de caracteres apontada pelo parâmetro *str*, iterando sobre o vetor e enviando cada caractere sucessivamente por meio de `uart_putc()`. Ao encontrar o caractere `\n`, finaliza a transmissão.

char uart_getc(void) Função sem parâmetros que retorna um caractere que foi recebido e transmitido pela UART. Ela funciona como uma interface pública de leitura do que a UART transmite.

void uart_irq_handler(void) Deve ser acionada diretamente pelo controlador de interrupções da arquitetura (GIC) sempre que o identificador de interrupção correspondente (*IRQ ID* 33) for disparado pelo hardware da UART. Inicia uma rotina de tratamentos de interrupções de UART.

2.4.2 API de Entrada e Saída do Kernel (kstdio)

Implementada sobre o módulo `kstdio.c`, esta interface fornece abstrações de mais alto nível para a comunicação textual e o processamento de comandos do sistema, encapsulando as complexidades da controladora UART. Suas funções compreendem:

void kstdio_init(void) Inicializa o subsistema básico de entrada e saída do *kernel*, acionando internamente a rotina de configuração da UART.

void kputs(const char *str) Envia uma *string* completa (recebida através de um ponteiro) para a saída serial, servindo como interface de alto nível para exibição de mensagens estáticas do *kernel*.

void kputc(char c) Envia um único caractere para a saída serial utilizando as rotinas de abstração de E/S.

char kgetc(void) Função sem parâmetros que retorna um caractere que foi recebido e transmitido pela UART. Ela funciona como uma interface pública de leitura do que a UART transmite.

char kgets(char *buf, int max_len) Lê uma sequência de caracteres da entrada padrão até encontrar uma quebra de linha ou atingir o limite de comprimento especificado em `max_len`, armazenando o resultado no vetor de destino *buf*.

void kprintf(const char *format, ...) Implementação personalizada de formatação e impressão para o *kernel*, inspirada na clássica função **printf()** da biblioteca padrão de C. Interpreta uma *string* de formatação contendo especificadores, convertendo argumentos subsequentes para representação textual e direcionando-os à saída serial. Os especificadores de formato suportados estão listados na Tabela 2.3.

Especificador	Descrição
%c	Caractere isolado
%s	Cadeia de caracteres (<i>string</i>)
%d	Número inteiro decimal com sinal
%u	Número inteiro decimal sem sinal
%x	Representação em formato hexadecimal
%n f	Número de ponto flutuante, formatado com <i>n</i> casas decimais
%%	Caractere literal de porcentagem (%)

Tabela 2.3: Especificadores de formato suportados por **kprintf()**.

2.5 Testes e Verificação

Para validar o funcionamento integrado de todo o subsistema de E/S, utilizou-se o interpretador de comandos (*shell*) implementado no arquivo **shell.c**. O *shell* provê uma interface de linha de comando simples para interação direta com o *kernel* através da porta serial.

O fluxo de execução do *shell* baseia-se em um laço contínuo que exibe um indicador visual (*prompt* \$) e aguarda a entrada do usuário por meio da função **kgetc()**. Os caracteres digitados são armazenados temporariamente em um *buffer* interno de comandos. Durante essa captura, o sistema trata dois caracteres especiais de controle, descritos na Tabela 2.4.

Tecla	Efeito
<i>Enter</i>	Finaliza a digitação da linha atual, sinalizando que o comando está pronto para o processamento.
<i>Backspace</i>	Remove o último caractere inserido, atualizando visualmente o terminal.

Tabela 2.4: Tratamento de teclas especiais pelo *shell*.

Para garantir a interatividade, cada caractere imprimível digitado é imediatamente enviado de volta ao terminal através de **kputc()**, gerando o efeito de eco visual. Após a conclusão da linha, o *shell* executa uma etapa de *parser* (tokenização), que divide a entrada com base em espaços, preenchendo o vetor de argumentos **argv** e atualizando a contagem em **argc**.

O reconhecimento e a execução dos comandos são efetuados por meio de comparações de *strings* utilizando a função **strcmp()**. O conjunto de comandos implementados, descrito na Tabela 2.5, permitiu testar exaustivamente as rotinas do sistema.

Comando	Função e Cobertura de Teste
<code>help</code>	Exibe a listagem de comandos disponíveis, testando a capacidade de impressão de múltiplas linhas formatadas via <code>kprintf()</code> .
<code>clear</code>	Limpa o terminal serial enviando a sequência de escape ANSI padrão (<code>\033[2J\033[H</code>) por meio da função <code>kputs()</code> .
<code>echo</code>	Reproduz no terminal os argumentos subsequentes informados pelo usuário. Valida a leitura integrada do <i>buffer</i> , a tokenização de <i>strings</i> e a transmissão formatada de texto.
<code>kprintf</code>	Executa uma bateria de testes da função de impressão formatada, validando a conversão e exibição de múltiplos especificadores (<code>%s</code> , <code>%c</code> , <code>%d</code> e <code>%x</code>).

Tabela 2.5: Comandos do *shell* utilizados na verificação do subsistema de E/S.

Memória

Autor(es): *Felipe da Rocha, Felipe Ferreira e Pedro Augusto*

3.1 Descrição

O módulo de memória é responsável pelo gerenciamento da memória física e virtual do sistema operacional. Ele fornece mecanismos para controle das páginas disponíveis na RAM, alocação de memória para outros módulos do kernel e configuração da Unidade de Gerenciamento de Memória (MMU).

A implementação foi dividida em três componentes principais:

- **PMM (Physical Memory Manager):** responsável pelo gerenciamento da memória física através de um bitmap.
- **VMM (Virtual Memory Manager):** responsável pela criação das tabelas de páginas e ativação da MMU.
- **API de Memória:** interface utilizada pelos demais módulos para solicitar e liberar memória.

Essa divisão permite separar responsabilidades e simplificar futuras expansões do sistema.

3.2 Decisões de Arquitetura e Design

Para o gerenciamento da memória física foi escolhido o uso de um **bitmap de páginas**. Nesse modelo, cada bit representa uma página de memória de 4 KB:

- Bit 0: página livre.
- Bit 1: página ocupada.

Outras alternativas consideradas foram listas encadeadas de blocos livres. Entretanto, o bitmap foi escolhido devido à sua simplicidade de implementação e ao baixo consumo de memória, características importantes para um kernel educacional.

O bitmap é armazenado logo após o final da imagem do kernel na memória. Durante a inicialização, todas as páginas são marcadas como ocupadas e, posteriormente, as regiões livres são liberadas.

Para o gerenciamento virtual foi adotado o esquema de paginação, composto por:

- Diretório de páginas de primeiro nível (L1);
- Tabelas de páginas de segundo nível (L2);
- Mapeamento identidade (virtual = físico) durante a inicialização.

O mapeamento identidade foi utilizado para simplificar a ativação da MMU, garantindo que o kernel continue executando normalmente após a mudança para memória virtual.

Como limitação, o sistema ainda não implementa paginação por demanda, swapping ou algoritmos avançados de substituição de páginas.

3.3 Detalhes da Implementação

3.3.1 Gerenciador de Memória Física (PMM)

A estrutura principal utilizada é:

```
1 typedef struct {  
2     uint32_t *map;  
3     uint32_t size;  
4 } Bitmap;
```

Onde:

- **map**: vetor contendo os bits que representam as páginas.
- **size**: quantidade de inteiros utilizados pelo bitmap.

Inicialização

A função `pmm_init()` configura o bitmap e libera as páginas disponíveis após o kernel. Fluxo de execução:

1. Posiciona o bitmap após o final da imagem do kernel.
2. Calcula o número total de páginas da RAM.
3. Marca todas as páginas como ocupadas.
4. Calcula o primeiro endereço livre.
5. Libera todas as páginas até o final da RAM.

Liberação de Página

```
1 void pmm_free_page(uintptr_t addr)  
2 {  
3     ...  
4     bitmap.map[idx] &= ~mask;  
5 }
```

A operação realiza um AND com a máscara invertida, fazendo com que o bit correspondente seja zerado.

Alocação de Página

```
1 void* pmm_alloc_block()  
2 {  
3     ...  
4     bitmap.map[i] |= mask;  
5     return (void*)phys_addr;  
6 }
```

O algoritmo percorre o bitmap procurando o primeiro bit livre e o marca como ocupado.

3.3.2 Gerenciador de Memória Virtual (VMM)

A função `vmm_init()` realiza a configuração da memória virtual. As etapas executadas são:

1. Inicialização do diretório de páginas.
2. Mapeamento da UART.
3. Mapeamento identidade de toda a RAM.
4. Carregamento do diretório no registrador TTBR0.
5. Ativação da MMU.

Mapeamento de Página

A função `vmm_map_page()` cria ou reutiliza uma tabela L2 e insere uma entrada apontando para a página física desejada.

```
1 pt->entries[pt_idx] =  
2 (physical_addr & 0xFFFFF000)  
3 | flags  
4 | 0x2;
```

Ativação da MMU

```
1 void vmm_enable()  
2 {  
3 ...  
4 }
```

A função `vmm_enable()` realiza:

- Configuração do DACR;
- Leitura do registrador SCTLR;
- Habilitação do bit de ativação da MMU;
- Limpeza do pipeline através da instrução ISB.

3.3.3 API de Memória

A API fornece uma interface simplificada para os demais módulos. Atualmente estão implementadas:

- `kmalloc()` – reserva memória.
- `kfree()` – libera memória.

A implementação de `kmalloc()` ainda encontra-se em desenvolvimento e atualmente calcula o número de páginas necessárias para a alocação.

3.4 Interface Pública (API)

3.4.1 API de Uso Geral (Gerenciamento de Heap)

Esta subseção documenta as funções de alto nível voltadas para a alocação dinâmica de memória no kernel. Esta é a interface padrão exposta e visível para a grande maioria dos outros subsistemas do kernel (como sistemas de arquivos, gerência de rede e drivers comuns de dispositivos).

kmalloc

Descrição: Solicita memória dinâmica ao kernel.

Parâmetros:

- **size:** quantidade de bytes desejada.

Retorno: Endereço da memória virtual alocada no Heap.

kfree

Descrição: Libera uma área de memória anteriormente reservada no Heap.

Parâmetros:

- **addr:** endereço de memória a ser liberado.

Retorno:

- 1 em caso de sucesso.

3.4.2 API de Baixo Nível / Core (Subsistemas Críticos)

Esta subseção documenta as rotinas de controle direto sobre a memória física (PMM) e virtual (VMM). O acesso a estas funções é restrito e especializado, sendo direcionado principalmente para o código de boot do sistema (**kmain**), o gerenciador de processos (scheduler) para criação de novos contextos/espacos de endereçamento, e drivers de baixo nível que exigem mapeamento direto de registradores de hardware (MMIO).

pmm_init

Descrição: Inicializa o gerenciador de memória física e configura a estrutura interna do bitmap. Deve ser invocada estritamente durante a rotina de boot.

Parâmetros: Nenhum.

Retorno: Não possui retorno.

pmm_alloc_block

Descrição: Procura e aloca uma página física livre diretamente da RAM.

Parâmetros: Nenhum.

Retorno:

- Endereço da página física alocada.
- NULL caso não exista memória livre.

pmm_free_page

Descrição: Libera uma página física previamente alocada, devolvendo-a para o bitmap de páginas livres.

Parâmetros:

- **addr:** endereço físico da página.

Retorno: Não possui retorno.

vmm_init

Descrição: Inicializa a estrutura base do sistema de memória virtual e ativa a MMU do processador. Deve ser invocada apenas uma vez no boot.

Parâmetros: Nenhum.

Retorno: Não possui retorno.

vmm_map_page

Descrição: Mapeia explicitamente uma página virtual para uma página física correspondente no diretório de tabelas. Útil para mapeamento de hardware (MMIO) e isolamento de processos.

Parâmetros:

- **dir:** diretório de páginas (L1).
- **virtual_addr:** endereço virtual desejado.
- **physical_addr:** endereço físico de destino.
- **flags:** permissões e flags de acesso à página (ex: leitura, escrita, cache).

Retorno:

- 0 em caso de sucesso.
- -1 em caso de erro.

3.5 Testes e Verificação

Os testes foram executados dentro do ambiente QEMU utilizando a porta serial para exibição das mensagens de depuração.

A rotina de testes foi adicionada à função `kmain()`.

Durante o boot são executadas as seguintes etapas:

1. Inicialização da serial.
2. Inicialização do PMM.
3. Teste da API de memória.
4. Inicialização do VMM.
5. Ativação da MMU.

Saída esperada:

```
--- KERNEL BOOT: TESTE DE MEMORIA ---  
[INIT] Inicializando PMM...  
[TEST] Testando kmalloc...  
[TEST] kmalloc alocou memoria com sucesso!  
[INIT] Inicializando VMM e ativando MMU...  
[SUCCESS] MMU ativa e rodando em modo mapeado!
```

Os seguintes casos foram validados:

- Inicialização correta do bitmap.
- Liberação das páginas após o kernel.
- Alocação de páginas físicas livres.
- Criação dinâmica de tabelas de páginas.
- Mapeamento identidade da RAM.
- Ativação da MMU sem geração de exceções.

Os resultados obtidos foram compatíveis com o comportamento esperado, demonstrando o funcionamento correto da infraestrutura básica de gerenciamento de memória do sistema operacional.

Chamadas de Sistema

Autor(es): *Cauã Marques da Silva, Felipe de Castro Duchon Aurox, Felipe de Oliveira Renó, Vicente Pinto Tomás Junior*

4.1 Descrição

As chamadas de sistemas são o mecanismo pelo qual o sistema operacional fornece aos processos acesso aos recursos do sistema. Essencialmente, as chamadas de sistema são a forma pela qual os processos têm acesso aos arquivos, diretórios, ao relógio, teclado e até mesmo outros processos.

O sistema operacional oferece esse mecanismo para poder manter a separação entre os dois principais modos de operação do processador, os modos supervisor e usuário. No modo supervisor o processador pode executar qualquer instrução e interagir com outros componentes físicos do sistema, já no modo usuário o processador está restrito a apenas um subconjunto dessas instruções e não pode interagir com outros componentes físicos do sistema. Esse isolamento busca impedir que um processo, defeituoso ou malicioso, assuma controle dos recursos do sistema.

Além disso, a interface criada por esse mecanismo oferece aos programas uma abstração dos recursos do sistema, ou seja, um programa não precisa conhecer o hardware responsável pela memória se ele deseja salvar um arquivo, basta que ele peça ao sistema operacional. Ademais, o programa não precisa nem saber como a informação será efetivamente salva na memória, ele apenas precisa fornecer o que ele deseja salvar e o sistema operacional lidará com o resto.

4.2 Decisões de Arquitetura e Design

As chamadas de sistema estão diretamente ligadas a forma como o núcleo é estruturado, na literatura discutem-se, principalmente, duas arquiteturas: núcleo monolítico e micronúcleo.

A arquitetura monolítica, essencialmente, se baseia em concentrar as operações de nível de sistema dentro do núcleo, ou seja, a gerência de memória, o sistema de arquivos, as pilhas dos protocolos de rede e outros existem no espaço do núcleo (são executados em modo supervisor).

Na arquitetura micronúcleo as operações de nível de sistema são transferidas para o espaço do usuário (são executadas em modo usuário) por módulos próprios separados do núcleo. O núcleo fica, portanto, responsável por escalonar e gerenciar a comunicação entre os processos que agora, também, incluem os módulos do sistema.

Dessa forma, as chamadas de sistema em um núcleo monolítico são executadas em modo supervisor e interagem diretamente com os recursos do sistema e em um micronúcleo as chamadas seriam transformadas em trocas de mensagens entre o processo fazendo a chamada e o processo responsável pelo recurso do sistema, de forma que o micronúcleo funcionaria apenas como um mediador encaminhando essas mensagens.

Nesse contexto, para o desenvolvimento das chamadas de sistema neste projeto, adotou-se uma abordagem de núcleo monolítico visando simplificar os detalhes de implementação das chamadas em si, uma vez que as mesmas podem interagir diretamente com os recursos do sistema. Consequentemente, essa abordagem acarreta em um núcleo mais complexo (no que

se refere a quantidade de partes e responsabilidades do núcleo), o que reduz a modularidade e introduz mais pontos de falha no sistema (uma vez que as chamadas de sistema acontecem em modo supervisor o que pode ser perigoso caso haja falhas no código).

4.3 Detalhes da Implementação

Há quatro principais componentes envolvidos em realizar uma chamada de sistema, sendo eles: o invocador, o tratador SVC, o tratador de chamada de sistema e a tabela de chamadas de sistema.

Os invocadores são as rotinas encapsuladas pelas funções em C que oferecem as chamadas de sistema para os processos. Essas precisam registrar o ponto atual da execução (para que se possa retornar a esse ponto posteriormente), registrar o endereço da rotina que será executada e, então, gerar uma exceção forçando a transferência da execução para o sistema operacional em modo supervisor.

```
1 exemplo:
2     push {lr}
3     mov r0, #CHAMADA
4     SVC #0
5     pop {pc}
```

Listing 4.1: Exemplo de invocador

Quando a instrução SVC é usada, o processador gera uma exceção, ele então consulta a tabela de exceções e transfere a execução para a rotina identificada na tabela. O tratador SVC, simplesmente, salva os registradores comuns e transfere a execução para o tratador de chamada de sistemas.

```
1 svc_handler:
2     STMFD sp!, {r1-r12, lr}
3     bl syscall_Handler
4     LDMFD sp!, {r1-r12, pc}~
```

Listing 4.2: Tratador SVC

O tratador de chamadas de sistema interpretará pelos argumentos da chamada qual é a intenção do processo e fará uma consulta na tabela de chamadas de sistema.

```
1 syscall_Handler:
2     ldr r3, =Syscall_Table
3     ldr r3, [r3,r0,LSL #2]
4     bx r3
```

Listing 4.3: Tratador de chamada de sistema

Se possível, ele executará a rotina associada ao endereço na tabela e devolverá o resultado e a execução para o processo que fez a chamada de sistema.

```
1 Syscall_Table:
2     .word printf_handler
3     .word abort_handler
```

Listing 4.4: Tabela dos endereços das chamadas de sistema

```
1 printf_handler:
2     mov r0,r1
3     push {lr}
4     bl serial_puts
5     mov r0, #10
6     pop {pc}
```

Listing 4.5: Exemplo de rotina que será de fato executada

4.4 Interface Pública (API)

4.4.1 printf

Imprime o texto passado como parâmetro na saída serial.

Parâmetros: um ponteiro de char apontando para o texto a ser impresso.

Valor de retorno: um inteiro indicando se o print foi realizado com sucesso (retorno 0) ou não (retorno -1).

4.4.2 abort

Força o reinício da CPU. Em processadores ARM é necessário interagir com o PSCI (*Power State Coordination Interface*).

Valor de retorno: um inteiro indicando se o abort foi definido com sucesso (retorno 0) ou não (retorno -1).

Observação: por enquanto a chamada de sistema não possui uma forma de retornar nenhum valor uma vez que o reboot é instantâneo.

4.5 Testes e Verificação

As rotinas que invocam as chamadas de sistema estão encapsuladas em C, então os testes consistiram em invocá-las, verificar, pelo depurador, as alterações no fluxo de execução e observar o resultado da execução das rotinas.

Para a chamada *printf*, a saída esperada era a mensagem "Testando alterações" exibida na tela e a saída obtida foi a mensagem "Testando alterações" exibida na tela.

Para a chamada *abort*, era esperado o reinício da CPU com a mensagem "Executando em modo ARM bare-metal no QEMU." sendo exibida na tela e o resultado obtido foi um salto do endereço corrente da memória para o endereço *start* (sinalizando o reinício) e subsequentemente a mensagem "Executando em modo ARM bare-metal no QEMU." exibida na tela.

Essas chamadas de sistema apenas encapsulam rotinas que já existiam (no caso do *printf*) ou instruções especiais da CPU (no caso do *abort*), portanto os casos de borda se restringiram ao que acontece durante uma chamada de sistema, ou seja, se os registradores estavam sendo manipulados corretamente (salvos e depois restaurados), o que foi constatado pela inspeção durante a depuração de ambas as chamadas.

Processos, Troca de Contexto e Escalonamento

Autor(es): *João Pedro Rizzo Sales, Samuel Romano Ribeiro, Guilherme Michelsen do Monte*

5.1 Descrição

Este capítulo documenta três módulos do UFSKernel: o módulo de **processos**, responsável por representar, criar e encerrar unidades de execução; o mecanismo de **troca de contexto**, que realiza a alternância segura entre modo usuário e modo kernel; e o **escalonador**, que decide qual processo recebe a CPU e por quanto tempo. Juntos, esses módulos viabilizam a multiprogramação no kernel, conforme descrito em Silberschatz et al. [6] e Tanenbaum e Bos [7].

Os arquivos que compõem esses módulos são: `process.h`, `process.c`, `fork.c`, `exec.c`, `exit.c`, `proc_ll.c`, `svc_entry.S`, `irq_entry.S`, `irq_return.S`, `context_switch.S`, `scheduler.h` e `process_control_block.c`.

5.2 Fundamentação Teórica

5.2.1 Processo e PCB

Um processo é um programa em execução: uma entidade dinâmica que engloba o contador de programa, os registradores, a pilha e o espaço de endereços ocupado na memória [7]. Para gerenciá-los, o kernel mantém um **Bloco de Controle de Processo** (PCB) por processo, concentrando tudo o que é necessário para suspendê-lo e retomá-lo como se nunca tivesse sido interrompido [6].

Durante seu ciclo de vida, um processo transita entre quatro estados. Os três canônicos, **Pronto**, **Em execução** e **Bloqueado**, são descritos em [7]. O UFSKernel acrescenta um quarto, **Zumbi**: o processo encerrou, mas sua estrutura ainda não foi descartada, pois o processo pai deve registrar o encerramento do filho antes que os recursos sejam liberados [6].

5.2.2 Trapframe, Contexto de Kernel e ABI ARM

Na ARMv7, o processador opera em modos de privilégio distintos. Toda transição de modo usuário para kernel, seja por chamada de sistema (SVC) ou por interrupção (IRQ), exige salvar o estado completo do processo para que a execução possa ser retomada. A estrutura que armazena esse estado é o **trapframe** [7].

Para trocas de contexto internas ao kernel, quando o escalonador substitui um processo por outro sem sair do modo supervisor, apenas um subconjunto dos registradores precisa ser salvo. A ABI ARM (AAPCS) [arm2014] define dois grupos: registradores *call-preserved* (r4–r11, SP e LR), que uma função deve restaurar antes de retornar, e registradores *call-clobbered* (r0–r3, r12), que o chamador sabe que podem ser destruídos. Como o compilador já protege os

call-clobbered em torno de qualquer chamada de função, o mecanismo de troca de contexto de kernel precisa salvar apenas os call-preserved.

5.2.3 Escalonamento: Round-Robin e MLFQ

O escalonador decide qual processo pronto recebe a CPU e por quanto tempo (o **quantum**), tipicamente acionado por um timer periódico [6]. O **Round-Robin** organiza os processos em fila circular com quantum fixo, mas trata igualmente processos interativos e CPU-bound [7]. A **Multilevel Feedback Queue** (MLFQ) supera essa limitação: mantém múltiplas filas de prioridade com quanta crescentes, promovendo processos que usam pouco CPU (interativos) e rebaixando os que esgotam o quantum (CPU-bound), de modo que cada processo migra naturalmente para o nível compatível com seu comportamento real [6].

5.3 Módulo de Processos

5.3.1 Decisões de Arquitetura e Design

O módulo mantém duas estruturas distintas para o estado de execução de um processo. O `struct trapframe` armazena todos os registradores de propósito geral (r0–r12), os registradores banked do modo usuário (`sp_usr`, `lr_usr`), o contador de programa (`pc_usr`) e o registrador de estado (`cpsr_usr`) — tudo o que o processo de usuário enxerga. O `struct context` armazena apenas os registradores call-preserved (r4–r11, SP e LR), suficientes para a troca de contexto dentro do kernel. Salvar r0–r3 e r12 no contexto de kernel seria redundante, pois o compilador já os trata como voláteis em torno de qualquer chamada [arm2014].

Alocação estática de memória

Sem um alocador dinâmico disponível, toda a memória dos processos foi provisionada em tempo de compilação com três arrays globais em `fork.c`:

Listing 5.1: Arrays de alocação estática em `fork.c`

```
1 struct process structProcessos[MAX_PROCESS_COUNT]; // 64 PCBs
2 char memProcessos[MAX_PROCESS_COUNT][SIZE_1MB]; // 64 x 1 MB
3 char kstackProcessos[MAX_PROCESS_COUNT][SIZE_16KB]; // 64 x 16 KB
```

Isso impõe limites fixos: máximo de 64 processos simultâneos, 1 MB de memória e 16 KB de pilha de kernel por processo, independentemente da necessidade.

Simulação de executáveis com `struct arquivo`

Sem um sistema de arquivos disponível, a noção de executável foi simulada pela `struct arquivo`: um ponteiro de função (`start`) e um tamanho (`size`).

Listing 5.2: Estrutura de simulação de arquivo em `process.h`

```
1 struct arquivo {
2     void (*start)(void); // funcao que representa o programa
3     int size; // tamanho do codigo a copiar
4 };
```

Isso permite testar o ciclo completo fork–exec–exit sem depender de sistema de arquivos, mas implica que o código do processo coexiste na mesma imagem binária do kernel, sem separação real de espaço de endereços.

Gestão de PIDs e a estrutura `process_block_list`

PIDs são gerenciados por `create_pid` e `free_pid` sobre o array circular `used_pids[MAX_PROCESS_COUNT]`. O PID 0 é reservado, convenção Unix que evita confundi-lo com um ponteiro nulo [6]. A estrutura `process_block_list`, definida em `process.h`, prevê filas de processos bloqueados por *sleep* ou por operações de E/S, mas não é utilizada na implementação atual, servindo como ponto de extensão planejado [7].

5.3.2 Detalhes da Implementação

O PCB: `struct process`

Listing 5.3: Bloco de Controle de Processo em `process.h`

```
1 struct process {
2     char* mem;           // inicio da regio de memoria do processo
3     unsigned msize;      // tamanho da regio de memoria
4     char* kstack;        // inicio da pilha de kernel
5     struct trapframe *tf; // ponteiro para o trapframe (offset 12)
6     enum State state;     // RUNNING, READY, BLOCKED ou ZOMBIE
7     enum Block_Reason blocked_by;
8     pid_t pid;
9     process parent;
10    pid_list children_ids;
11    struct context context; // registradores call-preserved (troca de
12    unsigned priority;
13 };
```

A ordem dos quatro primeiros campos é deliberada: `mem` (4 bytes), `msize` (4 bytes) e `kstack` (4 bytes) posicionam `tf` no offset 12 — valor codificado na constante `TRAPFRAME_OFFSET = 12` usada pelo código assembly para acessá-lo diretamente.

Ciclo de vida: `first_process`, `fork`, `exec` e `exit`

`first_process` cria o processo inicial: aloca a estrutura por indexação estática no PID gerado, insere o processo no escalonador, define `current`, posiciona o trapframe no topo da `kstack` e chama `exec`. Ao final, chama `irq_return` para entregar o controle ao modo usuário como se fosse um retorno de interrupção.

`sys_fork` copia byte a byte a memória e a `kstack` do pai para o filho, recalcula a posição do trapframe e altera os ponteiros de contexto do usuário (`pc_usr`, `lr_usr`, `sp_usr`) para o espaço de endereços do filho:

Listing 5.4: Translação de endereços no `sys_fork`

```
1 unsigned int pc_offset =
2     newprocess->parent->tf->pc_usr
3     - (unsigned int)newprocess->parent->mem;
4 newprocess->tf->pc_usr =
5     (unsigned int)newprocess->mem + pc_offset;
```

O pai recebe o PID do filho em `r0`; o filho recebe zero, convenção Unix que permite distinguir os dois após o retorno [6]. O `context.lr` do filho aponta para `irq_return`, garantindo que o primeiro `context_switch` para ele desembarque corretamente no retorno ao modo usuário.

`sys_exec` copia o conteúdo do pseudo-arquivo para a memória do processo e prepara um trapframe novo. O endereço de entrada é calculado como:

Listing 5.5: Cálculo do entry point em `sys_exec`

```
1 unsigned int entry_point =
2     (unsigned int) p->mem
3     + (((unsigned int)arquivo->start) & ~1U) - 0x40000000);
```

A operação `& ~1U` zera o bit menos significativo do ponteiro de função — necessário porque em Thumb esse bit é 1 para sinalizar o modo de instrução, mas não faz parte do endereço real [arm2014]. O `cpsr_usr` é configurado para 0x30: modo USR com Thumb ativo.

`sys_exit` remove o processo do escalonador, libera o PID, marca o estado como ZOMBIE e invoca `pcb_elect` para eleger imediatamente o próximo processo.

5.4 Mecanismo de Troca de Contexto

5.4.1 Três mecanismos: entrada, retorno e troca

O mecanismo de troca de contexto é composto por três partes complementares: a **entrada** (`svc_entry.S`, `irq_entry.S`), que captura o estado do processo de usuário e o armazena em um trapframe na pilha de kernel; o **retorno** (`irq_return.S`), que restaura esse estado e devolve o controle ao processo; e a **troca de contexto de kernel** (`context_switch.S`), que salva e restaura apenas os registradores call-preserved para alternar entre processos enquanto permanece em modo supervisor. Mantê-los separados permite ao escalonador decidir, a cada interrupção, se retorna ao mesmo processo ou realiza uma troca.

5.4.2 Entrada via SVC e IRQ

Ao receber um SVC ou IRQ, o hardware ARMv7 salva automaticamente o CPSR em SPSR e o endereço de retorno em LR [arm2014]. O problema é que r0–r12 ainda contêm valores do processo interrompido, mas é preciso um registrador livre para operar. A solução é gravar r0 e r1 em posições negativas da pilha IRQ antes de reutilizá-los:

Listing 5.6: Salvamento temporário e troca de pilha em `irq_entry.S`

```
1 str r0, [sp, #-4]           @ salva r0 temporariamente na pilha IRQ
2 str r1, [sp, #-8]           @ salva r1 temporariamente na pilha IRQ
3 mov r1, sp                  @ guarda o SP_irq para restaurar r0 e r1 depois
4
5 ldr r0, =current            @ carrega endereco da variavel global current
6 ldr r0, [r0]                @ r0 = ponteiro para o processo atual
7 ldr sp, [r0, #TRAPFRAME_OFFSET] @ troca pilha IRQ pela kstack do processo
8
9 ldr r0, [r1, #-4]           @ restaura r0
10 ldr r1, [r1, #-8]           @ restaura r1
```

Em seguida, o trapframe é construído na kstack do processo:

Listing 5.7: Construção do trapframe na pilha de kernel

```
1 sub sp, sp, #(17 * 4)       @ reserva espaco para os 17 campos do trapframe
2 stmia sp, {r0-r12}          @ salva r0-r12
3 add r12, sp, #(13*4)
4 stmia r12, {sp, lr}^        @ salva sp_usr e lr_usr (^ acessa banked USR)
5 str lr, [sp, #(15 * 4)]      @ salva PC (LR contem o end. de retorno do HW)
6 mrs r12, spsr
7 str r12, [sp, #(16 * 4)]     @ salva CPSR do processo (estava em SPSR)
```

O sufixo `^` em `stmia r12, {sp, lr}^` é essencial: ele instrui o processador a acessar os registradores banked do modo usuário (`SP_usr` e `LR_usr`), e não os do modo corrente. Sem ele, o estado do processo seria corrompido [arm2014].

5.4.3 Retorno ao Modo Usuário

`irq_return` realiza o processo inverso: lê `tf` do processo atual, restaura o CPSR do usuário no SPSR, carrega `r0-r12`, restaura `sp_usr` e `lr_usr` com o sufixo `^` e executa:

Listing 5.8: Retorno ao modo usuário em `irq_return.S`

```
1 movs pc, lr
```

O sufixo `s` com o PC como destino em modo privilegiado copia automaticamente o SPSR para o CPSR [arm2014] — restaurando modo, flags e conjunto de instruções em uma única instrução.

5.4.4 Troca de Contexto de Kernel

Listing 5.9: Troca de contexto de kernel em `context_switch.S`

```
1 context_switch:
2     cmp r0, #0
3     beq if_null           @ se nao ha processo saindo, apenas carrega
4
5     stmia r0!, {r4-r11}    @ salva registradores call-preserved
6     str sp, [r0], #4       @ salva SP
7     str lr, [r0]           @ salva LR (ponto de retorno quando reativado)
8
9 if_null:
10    ldmia r1!, {r4-r11}    @ restaura registradores call-preserved
11    ldr sp, [r1], #4       @ restaura SP (agora na pilha do novo processo)
12    ldr lr, [r1]           @ restaura LR
13
14    bx lr                 @ retorna -- no contexto do processo entrante
```

O caso `r0 == NULL` cobre o boot, quando não há processo saindo. A propriedade central é que `bx lr` não retorna para quem chamou `context_switch`, mas para o ponto em que o processo entrante havia sido suspenso. Para um processo recém-criado via `fork`, esse ponto é `irq_return`, colocado ali por `sys_fork`, fazendo-o ingressar diretamente no retorno ao modo usuário.

5.5 Escalonador

5.5.1 Decisões de Arquitetura e Design

MLFQ com 32 níveis e quanta crescentes

A ideia adotada é uma MLFQ simplificada com 32 níveis. O `pcb` é um array de `struct pcb_entry` com quanta variando de 20 a 175 em progressão aritmética de passo 5. Processos recém-criados entram no nível 0, maior prioridade, menor quantum, refletindo a heurística de que processos novos tendem a ser interativos [7]. A progressão aritmética produz separação gradual entre os níveis sem saltos abruptos que pudessem causar *starvation* nos níveis inferiores.

Margem de quantum e `list_location`

A constante `SCHEDULER_QUANTUM_MARGIN = 5` define uma zona de tolerância: sem ela, pequenas variações no tempo acumulado causariam oscilações de promoção e rebaixamento. A variável global `list_location`, atualizada por `pcb_get_node_pid`, comunica o nível atual do processo para `pcb_climb` e `pcb_fall`. É uma escolha que evita parâmetros extras, mas cria

acoplamento implícito: uma chamada intercalada a `pcb_get_node_pid` com outro PID sobrescreveria `list_location` e corromperia a movimentação. Em um sistema multiprocessado, isso seria uma fonte de *race conditions*.

Alocação estática e `time_elapsed`

Os nós da lista ligada são alocados estaticamente em `process_blocks_nodes[64]`, indexados por PID, impondo o mesmo limite de 64 processos. O comportamento do escalonador é funcional, mas o feedback de prioridade não reflete tempo real.

5.5.2 Detalhes da Implementação

Estruturas de dados

Listing 5.10: Estruturas do escalonador em `scheduler.h`

```
1 struct pcb_entry {
2     pll_node *process_list; // cabeca da lista ligada de processos
3     unsigned quantum;      // quantum deste nivel
4     short process_count;    // numero de processos no nivel
5     short next_process;     // indice circular do proximo a ser eleito
6 };
7
8 typedef struct process_node_ll {
9     struct process_node_ll *next;
10    process proc;
11 } pll_node;
```

O campo `next_process` implementa o revezamento circular dentro de cada nível: após eleger o processo de índice i , o campo avança para $(i + 1) \bmod \text{process_count}$.

O algoritmo de eleição: `pcb_elect`

Invocada a cada *tick* do timer via `timer_callback`, `pcb_elect` executa em três fases:

Fase 1 — Acumular tempo. Adiciona o retorno de `time_elapsed` à variável `running_for`.

Fase 2 — Avaliar o processo atual. Se o processo ainda está dentro do quantum (com a margem), retorna sem preemptar:

Listing 5.11: Avaliação do quantum em `pcb_elect`

```
1 if (current->state == RUNNING &&
2     running_for < (quantum + SCHEDULER_QUANTUM_MARGIN))
3     return; // ainda dentro do quantum, nao preempta
4
5 // esgotou o quantum: aplica feedback de prioridade
6 if (running_for <= quantum - SCHEDULER_QUANTUM_MARGIN)
7     pcb_climb(current_process_node); // usou pouco: promove
8 else
9     pcb_fall(current_process_node); // usou muito: rebaixa
```

Fase 3 — Eleger o próximo. Percorre os 32 níveis do mais alto para o mais baixo. Em cada nível, localiza o processo na posição `next_process` e avança enquanto o processo não estiver `READY`. Ao encontrar um candidato, atualiza `current`, marca-o como `RUNNING`, zera `running_for` e chama `context_switch`. O `goto end` permite abandonar o laço interno sem flags booleanas adicionais.

Promoção e rebaixamento

`pcb_climb` e `pcb_fall` movem o processo um nível acima ou abaixo, respectivamente, removendo o nó do nível atual e inserindo-o no adjacente. Os limites (nível 0 e nível 31) são respeitados: processos nos extremos não são movidos.

5.6 Interface Pública (API)

5.6.1 Módulo de Processos (`process.h`)

`pid_t create_pid()` Aloca e retorna um PID disponível. Retorna `MAX_PROCESS_COUNT` se todos estiverem em uso.

`void free_pid(pid_t pid)` Libera o PID para reutilização.

`void first_process(struct arquivo arquivo)` Cria e lança o processo inicial. Deve ser chamada uma única vez pelo `kmain`. Não retorna.

`int fork()` Cria um processo filho. Retorna o PID do filho ao pai e 0 ao filho (wrapper assembly em `syscall.S`).

`void exec(struct arquivo* arquivo)` Substitui a imagem do processo atual. Não retorna ao chamador.

`void exit()` Encerra o processo atual. Não retorna.

5.6.2 Módulo do Escalonador (`scheduler.h`)

`void pcb_select(void)` Seleciona e ativa o próximo processo. Chamada automaticamente pelo timer; pode ser invocada explicitamente por `sys_exit`.

`void pcb_insert(unsigned priority_level, pll_node* proc)` Insere um processo no nível especificado. Nível 0 é o de maior prioridade.

`void pcb_remove(unsigned priority_level, pll_node* proc)` Remove um processo do nível especificado.

`pll_node* pcb_get_node_pid(pid_t pid)` Localiza o nó do processo pelo PID. Efeito colateral: atualiza `list_location`.

`void pcb_climb(pll_node* proc)` Move o processo um nível acima. Requer chamada prévia de `pcb_get_node_pid` para o mesmo processo.

`void pcb_fall(pll_node* proc)` Move o processo um nível abaixo. Mesma condição de `pcb_climb`.

`pll_node* pll_node_new(process proc)` Inicializa e retorna o nó de lista ligada alocado estaticamente para o processo, indexado pelo PID.

`int proc_is_valid(process proc)` Retorna verdadeiro se o processo não é nulo e não está em estado ZOMBIE.

5.7 Testes e Verificação

5.7.1 Rotina de Teste

O `kmain` chama `first_process(arqinicio)`, que executa `programainicio`:

1. Imprime "Processo rodando programa inicio".
2. Chama `fork`; o filho invoca `exec(&arq1)`, carregando `programa1`.
3. O pai chama `fork` novamente; o segundo filho invoca `exec(&arq2)`, carregando `programa2`.
4. O pai chama `exit`.

Os dois filhos entram em laço infinito, imprimindo suas mensagens com um delay de espera ativa. O escalonador alterna entre eles a cada *tick* do timer.

5.7.2 Saída Esperada

Listing 5.12: Saída serial esperada no QEMU

```
1 Bem-vindo ao UFSKernel!
2 Executando em modo ARM bare-metal no QEMU.
3 Preparando CPU (VBAR e Pilha IRQ)...
4 Configurando GIC...
5 Ligando interrupcoes...
6 Ativando Timer...
7 Kernel rodando na main...
8 Processo rodando programa inicio
9 Processo rodando programa 1
10 Processo rodando programa 1
11 Processo rodando programa 1
12 Processo rodando programa 1
13 Processo rodando programa 1
14 >>> TICK DO TIMER <<<
15 Processo rodando programa 2
16 Processo rodando programa 2
17 Processo rodando programa 2
18 Processo rodando programa 2
19 Processo rodando programa 2
20 >>> TICK DO TIMER <<<
21 Processo rodando programa 1
22 ...
```

A intercalação de `>>> TICK DO TIMER <<<` com as impressões dos processos confirma que o escalonador está sendo acionado corretamente e alternando entre os processos a cada quantum.

Tratador de Interrupções

Autor(es): *Renan Matulianes Arnaldo, Renan Suana Grothe Garcia, João Vitor Grisoto Kussaba*

6.1 Descrição

O módulo de tratador de interrupções de um Sistema Operacional (SO) é responsável por receber as interrupções geradas pelo *hardware* ou pelo *software* e executar a respectiva rotina de tratamento para o tipo de interrupção recebida.

As interrupções são sinais gerados pelo *hardware* ou *software* que indicam que algum evento precisa de atenção completa do processador para serem tratados corretamente. Essas interrupções podem ocorrer por meio de chamadas de sistema, instruções específicas, erros em tempo de execução, entre outras rotinas. Desse modo, o tratador de interrupções é um módulo essencial do *kernel* do SO, porque ele define as rotinas de tratamento para realizar cada uma das respostas apropriadas para os sinais de interrupção recebidos.

6.2 Decisões de Arquitetura e Design

Para a implementação do tratador de interrupções, foi utilizada uma abordagem similar ao SO Minix, que é a utilização de uma tabela de vetores de bits que endereça fisicamente na memória onde está localizado a rotina específica de tratamento de interrupção, onde cada entrada dessa tabela possui um tipo de interrupção endereçada pelo *offset* de bits enviados.

Essa abordagem foi escolhida para que o acesso à rotina de tratamento de interrupções possa ser feito de forma constante, aumentando a eficiência em que o tratamento de interrupções ocorre e reduzindo o período em que o processador mantém-se ocupado com a rotina de tratamento.

6.3 Detalhes da Implementação

Seguindo a linha de pensamento abordada, foi utilizado um vetor de registro de funções para a implementação dos handlers de interrupções, que utiliza de ponteiros de função para acessar as rotinas de tratamento de interrupção. Além disso, é utilizada uma *struct* denominada *Interrupts Service Routine* (isr) responsável por realizar o resgistro das funções que farão o tratamento de interrupções pelos dispositivos.

Para isso, são utilizadas algumas funções tem tem por objetivo a manipulação dos bits presentes nos registradores GIC (*Generic Interrupt Controller*) da arquitetura ARM (arquitetura selecionada para a execução do projeto), que são registradores divididos por blocos lógicos responsáveis por gerenciar, priorizar e rotear interrupções. Para a etapa realizada neste projeto, os blocos mais utilizados para a configuração de interrupções no núcleo do sistema operacional são o bloco distribuidor *Generic Interrupt Controller Distributor* (GICD) e o bloco de interface da CPU *Generic Interrupt Controller CPU Interface* (GICC).

Esses blocos são manipulados pelas funções *gic_enable_interrupt()*, responsável por habilitar e registrar interrupções e modificar os registradores GIC_ISENABLER para que permitam o envio da interrupção à CPU;

```

1 void gic_enable_interrupt(uint32_t irq) {
2     if (irq >= MAX_INTERRUPTS)
3         return;
4
5     // cada registrador tem 32 bits
6     uint32_t reg_index = irq / 32; // qual registrador
7     uint32_t bit_offset = irq % 32; // qual bit do registrador
8
9     GICD_ISENABLER[reg_index] |= (1 << bit_offset);
10 }

```

e *gic_disable_interrupt()*, análogo ao *gic_enable_interrupt()*, só que desativa a interrupção selecionada e modifica os registradores GICD_ICENABLER para sinalizar que a interrupção está desativada;

```

1 void gic_disable_interrupt(uint32_t irq) {
2     if (irq >= MAX_INTERRUPTS)
3         return;
4
5     uint32_t reg_index = irq / 32;
6     uint32_t bit_offset = irq % 32;
7
8     // No ICENABLER escrever 1 desliga a interrupcao
9     GICD_ICENABLER[reg_index] = (1 << bit_offset);
10 }

```

Desse modo, o fluxo principal de execução funciona da seguinte forma: Um gatilho de *hardware* altera os bits de informação que são lidos pelo tratador como um tipo *volatile uint32_t* (sendo o tipo *uint32_t* um *typedef* do tipo *unsigned int* da linguagem C), que indica ao compilador que o valor de bits pode ser mudado inesperadamente fora do controle ou da visibilidade da execução do tratador, e são levados aos registradores GIC;

```

1 // Registradores
2 #define GICD_CTLR (*(volatile uint32_t *) (GICD_BASE + 0x000))
3 #define GICC_CTLR (*(volatile uint32_t *) (GICC_BASE + 0x000))
4 #define GICC_PMR (*(volatile uint32_t *) (GICC_BASE + 0x004))
5 #define GICC_IAR (*(volatile uint32_t *) (GICC_BASE + 0x00C))
6 #define GICC_EOIR (*(volatile uint32_t *) (GICC_BASE + 0x010))

```

após isso, a informação é levada à tabela de vetores em linguagem de máquina, que desvia a execução do programa para ela;

vector_table:	// offset	interrupcao	modo de execucao
b _start	// 0x00	Reset	supervisor
b undefined_handler	// 0x04	Undefined	undefined
b svc_handler	// 0x08	SVC	supervisor
b .	// 0x0C	Prefetch	abort
b data_abort_handler	// 0x10	Data Abort	abort
b .	// 0x14	Hyp Trap	hypervisor
b irq_handler	// 0x18	IRQ	interrupt
b .	// 0x1C	FIQ	fast interrupt

depois, o endereço da função em linguagem de máquina *irq_handler* ajusta o registrador de retorno da arquitetura para o processo o qual estava em operação antes da interrupção ter sido gerada e empilha todos os registradores na pilha presente na memória do processador para salvar o contexto antes de ocorrer a troca para o núcleo do SO;

```

1 irq_handler:

```

```

2      sub lr, lr, #4
3
4      // Uma pseudo-instrucao que armazena os registradores de uso geral R0-
5      R12 e o LR (Link Register)
6      // e ao mesmo tempo atualiza o valor do stack pointer (sp) para o topo
7      da pilha apos o armazenamento
8      stmfid sp!, {r0-r12, lr}
9
10     blx irq_dispatcher_c // Salta para a funcao em C
11
12     // Uma pseudo-instrucao que restaura os registradores R0-R12 da pilha
13     // e ao mesmo tempo copia o valor antigo de LR direto para o program
14     counter (PC)
15     ldmfd sp!, {r0-r12, pc}~

```

uma vez que o tratador de interrupções está em execução, o registrador GICC_IAR é lido para que o tratador saiba que interrupção o chamou e se essa interrupção possui uma rotina de tratamento configurada para ela. Caso possua, o tratador faz o desvio para a rotina específica armazenada no vetor de ponteiros de funções *interrupt_handlers* e salva o identificador da rotina no registrador GICC_EOIR, que indica que a interrupção já foi servida;

```

1 void irq_dispatcher_c(void) {
2     uint32_t irq_id = GICC_IAR & 0x3FF; // le o identificador da interrupcao
3
4     // verifica se o indenficador nao e maior que o maximo de interrupcoes
5     configuradas e se existe uma rotina de tratamento armazenada no vetor
6     interrupt_handlers
7     if (irq_id < MAX_INTERRUPTS && interrupt_handlers[irq_id] != 0) {
8         interrupt_handlers[irq_id]();
9     }
10
11     GICC_EOIR = irq_id; // marca que a interrupcao ja foi servida
12 }

```

por fim, o tratador realiza um retorno para o contexto do processo anterior a sua chamada para que ele possa continuar com a sua execução.

6.4 Interface Pública (API)

Para a utilização e configuração do tratador de forma que seja possível configurar e gerenciar interrupções, a biblioteca desenvolvida possui as seguintes funções:

- `int register_interrupt_handler(uint32_t irq, isr_t handler)`

Recebe o identificador de interrupção *irq* e um ponteiro de função *handler* que aponta para a rotina de tratamento da interrupção. Essa função salva o ponteiro de função no vetor *interrupt_handlers* no índice *irq* caso o identificador recebido não ultrapasse o valor MAX_INTERRUPTS. Retorna 0 caso tenha sido possível salvar o *handler* senão -1.

- `void gic_enable_interrupt(uint_t irq)`

Recebe o identificador de interrupção *irq* e habilita o registrador GICD_ISENABLER correspondente caso identificador não ultrapasse o valor MAX_INTERRUPTS, habilitando-o para o tratamento de interrupção adequado.

- `void gic_disable_interrupt(uint_t irq)`

Recebe o identificador de interrupção *irq* e habilita o registrador GICD_ICENABLER correspondente caso identificador não ultrapasse o valor MAX_INTERRUPTS, desabilitando o tratamento de interrupção correspondente.

- `void gic_config_interrupt(uint32_t irq, int edge_triggered)` Recebe o identificador de interrupção *irq* e um sinal *edge_triggered* que indica se a interrupção é sensível à mudança de estado imediata (valor 1) ou somente após a estabilização do sinal recebido (valor 0) e configura o índice do identificador no registrador `GICD_ICFGR` correspondente.

6.5 Testes e Verificação

Para o teste da funcionalidade, foi implementado o tratador para a interrupção de timer no método principal do projeto *kmain()*, que tem por objetivo imprimir na tela uma mensagem de aviso que o processador sofreu uma interrupção de timer.

Para isso, na função *kmain()*, é realizada a inicialização do serial do projeto por meio da função *serial_init()*, depois a CPU é inicializada para receber as estruturas de tratamento de interrupção por meio da função *setup_core_for_irq*, depois de inicializadas as estruturas para o tratamento de interrupções, as interrupções de CPU são ativadas com a função *enable_cpu_interrupts()* e, por fim, é chamada a função de inicialização e configuração do timer *init_timer*. Após essas configurações, um laço infinito é definido que imprime uma mensagem no console do QEMU indicando que o processo está operando normalmente no processador, com um segundo laço que serve apenas para espaçar o tempo de impressão de uma mensagem para outra, até sofrer uma interrupção de timer.

```

1 void kmain(void) {
2     serial_init();
3     serial_puts("Bem-vindo ao UFSKernel!\n");
4     serial_puts("Executando em modo ARM bare-metal no QEMU.\n");
5
6     serial_puts("Preparando CPU (VBAR e Pilha IRQ)...\n");
7     setup_core_for_irq();
8
9     serial_puts("Configurando GIC...\n");
10    init_gic();
11
12    serial_puts("Ligando interrupcoes...\n");
13    enable_cpu_interrupts();
14
15    serial_puts("Ativando Timer...\n");
16    init_timer();
17
18    while (1) {
19        serial_puts("Kernel rodando na main...\n");
20        // Um delay apenas para nao flodar a tela
21        for (volatile int i = 0; i < 500000000; i++);
22    }
23 }
```

O inicializador do timer *init_timer()* lê a frequência de ciclo do hardware, divide a frequência de ciclo pelo valor `SYSTEM_HZ`, que marca a frequência das interrupções de timer, para indicar quantos ticks ele deve aguardar antes de realizar uma interrupção, inicializa o timer com o valor de ticks calculado, registra a interrupção pela função *register_interrupt_handler()* e habilita a interrupção pela função *gic_enable_interruption()*.

```

1 void init_timer(void) {
2     uint32_t hardware_freq = read_cntfrq();
3     // quantos ticks formam um quantum no sistema
4     ticks_p_q = hardware_freq / SYSTEM_HZ;
5
6     // inicializa o timer com ticks_p_q
7     __asm__ volatile("mcr p15, 0, %0, c14, c2, 0" :: "r"(ticks_p_q));
8 }
```

```

8  uint32_t ctl = 1;
9  __asm__ volatile("mcr p15, 0, %0, c14, c2, 1" ::"r"(ctl));
10
11 // Toda interrupcao deve se registrar no despachante do GIC
12 if (!register_interrupt_handler(TIMER_IRQ, timer_callback))
13     serial_puts("Registrando timer no GIC...\n");
14 else
15     serial_puts("Erro ao registrar timer no GIC!\n");
16
17 // ativando a interrupcao apos registrar
18 gic_enable_interrupt(TIMER_IRQ);
19 }

```

Para a rotina de testes da interrupção configurada, a interrupção de timer chama a função *timer_callback()*, que imprime uma pequena mensagem alertando que uma interrupção de timer ocorreu, exatamente como deveria, variando apenas caso o valor de `SYSTEM_HZ` for configurado de uma forma diferente da já definida para o processo.

```

1 // funcao que sera chamada quando a interrupcao for ativada
2 static void timer_callback(void) {
3     serial_puts(">>> TICK DO TIMER <<<\n");
4
5     // recarrega o timer
6     __asm__ volatile("mcr p15, 0, %0, c14, c2, 0" ::"r"(ticks_p_q));
7
8     // e aqui que o codigo fara a chamada para o
9     // escalonador
10 }

```

Bibliografia

- [1] ARM Limited. *ARM Architecture Reference Manual ARMv7-A and ARMv7-R edition*. Issue C.c, ID072014. ARM Limited. 2014.
- [2] QEMU Team. *QEMU version 8.1.92, Hardware-independent devices documentation*. Online. Disponível em: <https://www.qemu.org/docs/master/system/arm/virt.html>. 2024.
- [3] Ekasiswanto. *How QEMU Emulates UART*. Online. Disponível em: <https://ekasiswanto.wordpress.com/2021/06/11/how-qemu-emulates-uart/>. Acesso em: 4 mai. 2026. 2021.
- [4] S. Jankovic. *Using QEMU UART Interface*. Online. Disponível em: <https://www.mistra-solutions.com/page/qemu-uart-interface/>. Acesso em: 5 mai. 2026. 2025.
- [5] ARM Limited. *PrimeCell UART (PL011) Technical Reference Manual – Summary of Registers*. Online. Document ARM DDI 0183G. Disponível em: <https://developer.arm.com/documentation/ddi0183/g/programmers-model/summary-of-registers>. Acesso em: 27 abri.. 2026. 2007.
- [6] Abraham Silberschatz, Peter Baer Galvin e Greg Gagne. *Operating System Concepts*. 10^a ed. Wiley, 2018. ISBN: 978-1-119-32091-3.
- [7] Andrew S. Tanenbaum e Herbert Bos. *Modern Operating Systems*. 4^a ed. Pearson, 2015. ISBN: 978-0-13-359162-0.